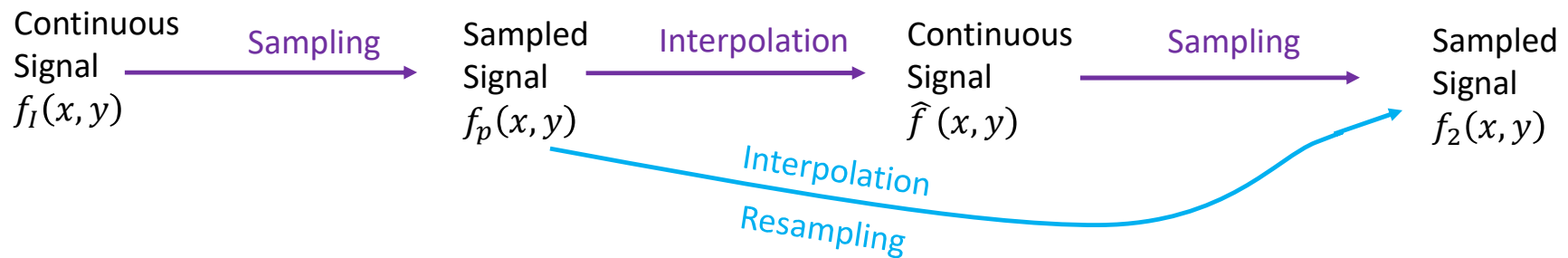


ECE 253 – Digital Image Processing

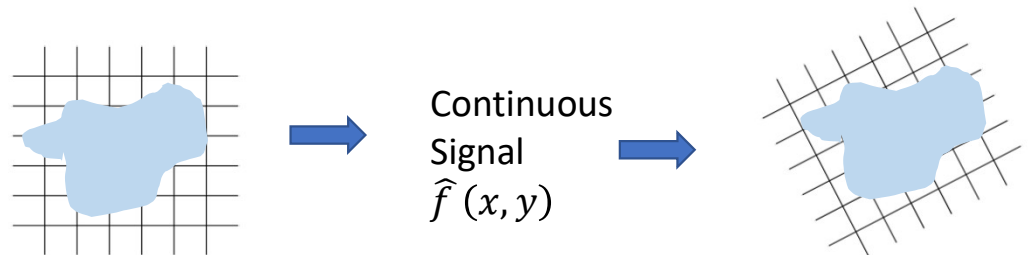
Sampling & Interpolation

Sampling and Interpolation in 2D



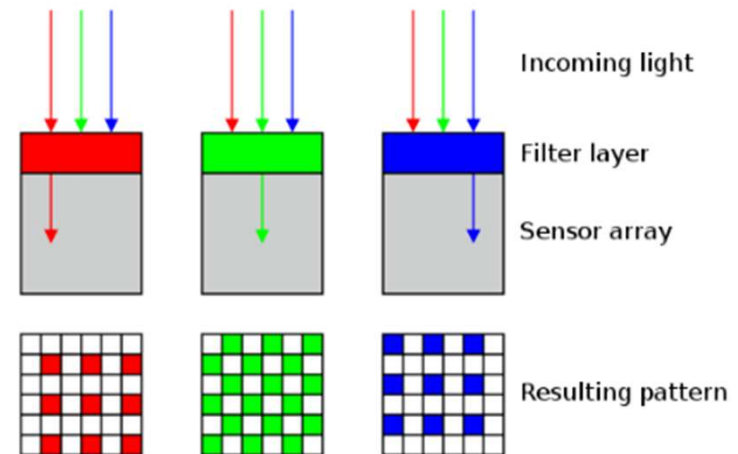
Where is interpolation used?

1. Need analog video (e.g., for CRT display)
2. Geometric transformations other than integer translations and 90° rotations
3. Filling in missing samples



Interpolation goals

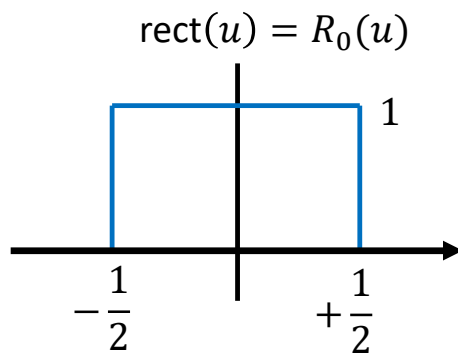
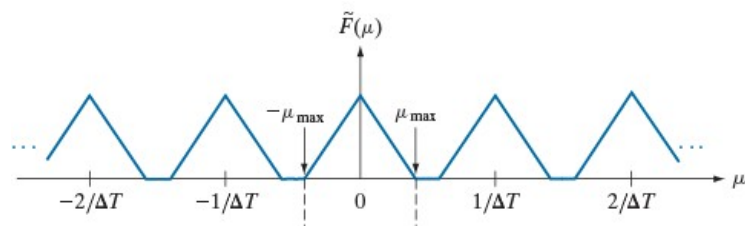
1. Geometric transformations other than integer translations and 90° rotations
 - Geometric corrections of an individual image
 - Registration (alignment) of one image to another
2. Filling in missing samples
 - JPEG/MPEG compression: RGB → YCbCr, downsample Cb,Cr
 - Bayer color filter array: Need to interpolate full resolution RGB arrays



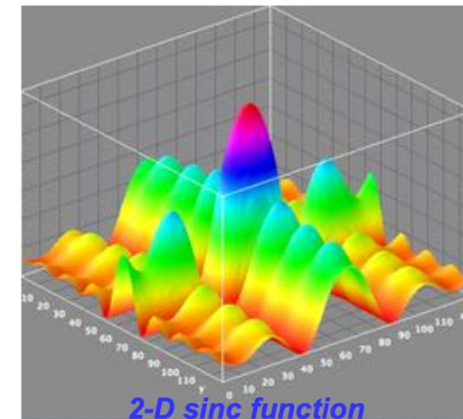
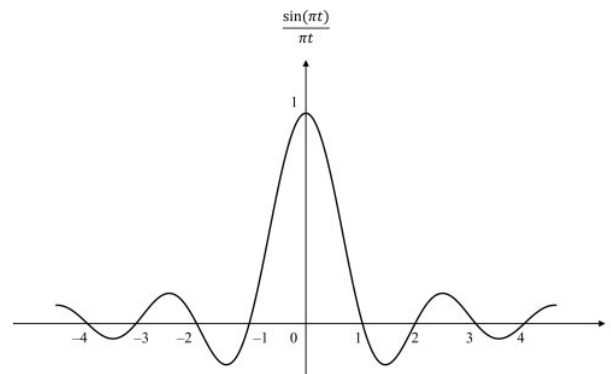
Ideal Interpolation

In ideal interpolation: In the frequency domain, we multiply spectrum by rect function (box)

Corresponds to convolution with a sinc function in the spatial domain



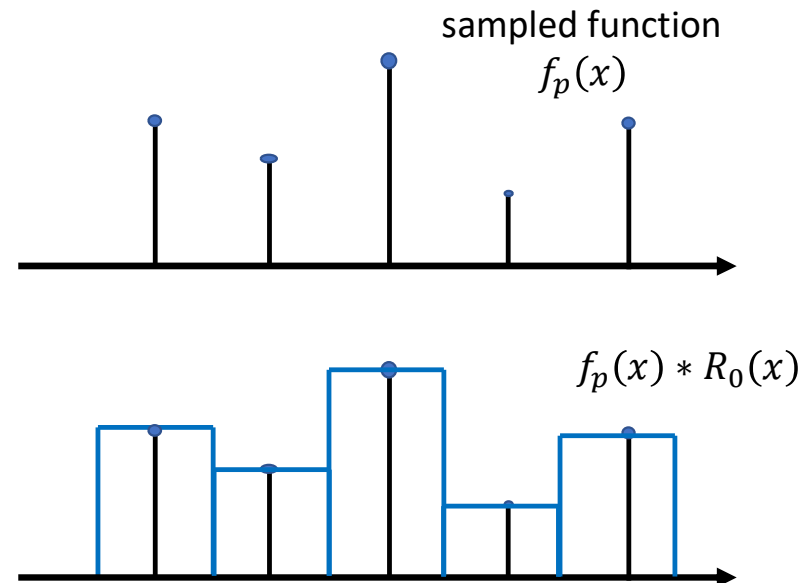
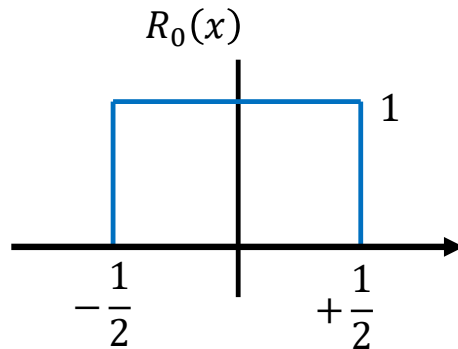
$$\text{sinc } x = \frac{\sin \pi x}{\pi x}$$



Sinc goes on forever → impossible to implement in practice → want to use convolution kernels that are reasonable computationally

Zero-order hold (also called pixel replication)

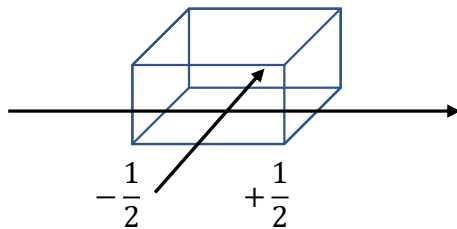
In 1D: Convolve with a square pulse



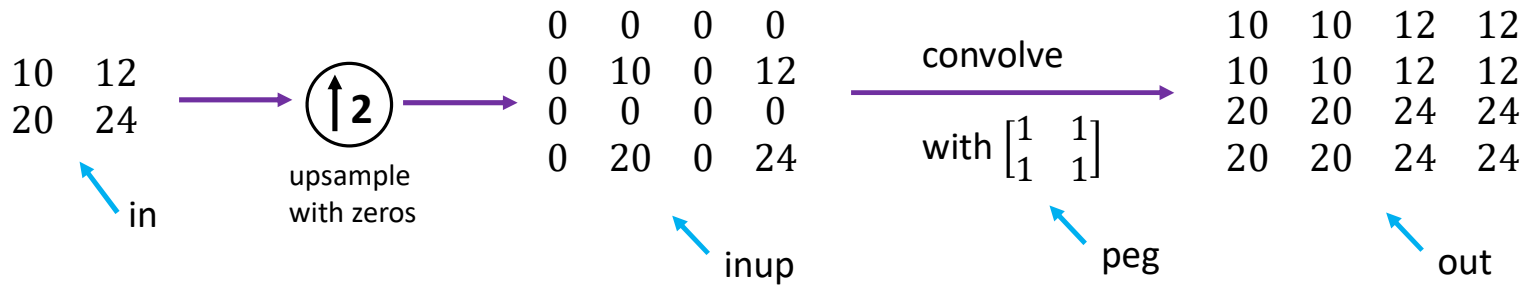
- The order of an interpolation scheme, by one definition, is one less than the # of points involved in defining an output value
- Need to specify that each sample “owns” the discontinuity to the right (so we can define resizing by a factor of 2, for example)

Zero-Order Hold in 2D (nearest neighbor interpolation, pixel replication)

Called Box filter or square peg filter



Upsampling an image with zeros by a factor of 2 involves inserting a row of zeros between adjacent image rows and a column of zeros between adjacent image columns



In Matlab:

- `out = conv2(inup, peg, 'same');`
- `out = imresize(in, 2, 'nearest');`

Downsampling by a factor of 2 involves deleting every other row & column



Zero order hold example



Original



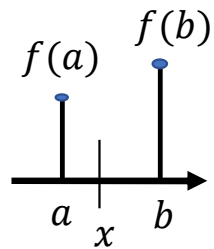
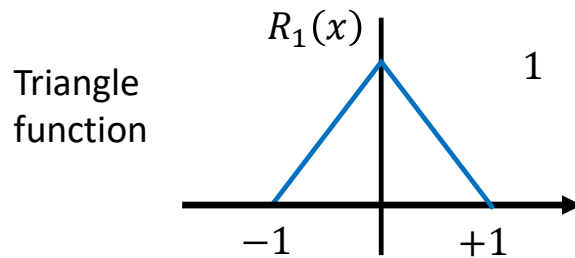
Sampled



Reconstructed

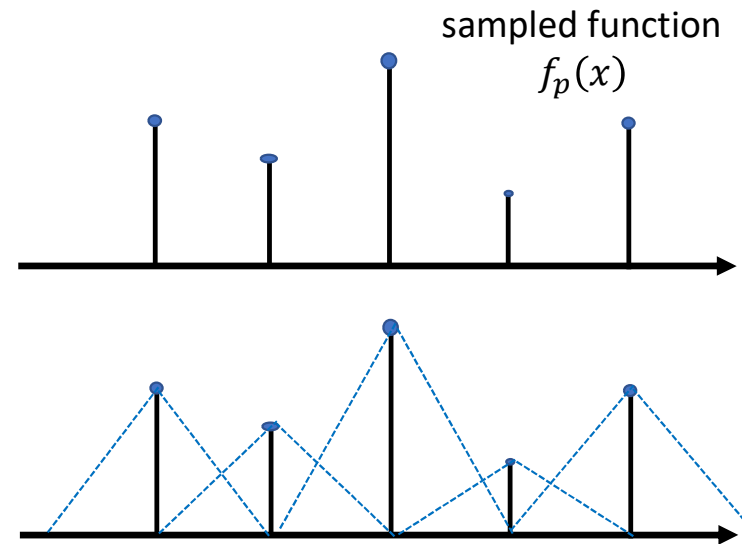
Linear Interpolation in 1D (first order hold)

$$R_1(x) = R_0(x) * R_0(x) = \begin{cases} 1 - |x| & \text{for } 0 \leq |x| < 1 \\ 0 & \text{elsewhere} \end{cases}$$

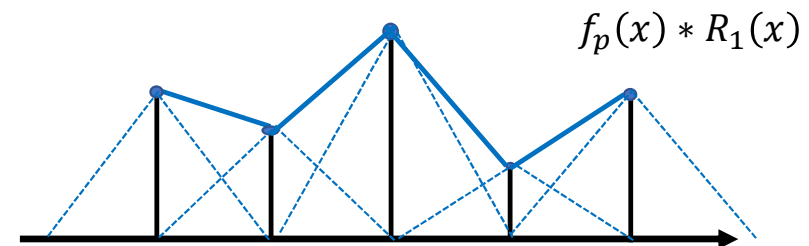


$$\Delta = \frac{x - a}{b - a}$$

$$\hat{f}(x) = \Delta f(b) + (1 - \Delta)f(a)$$

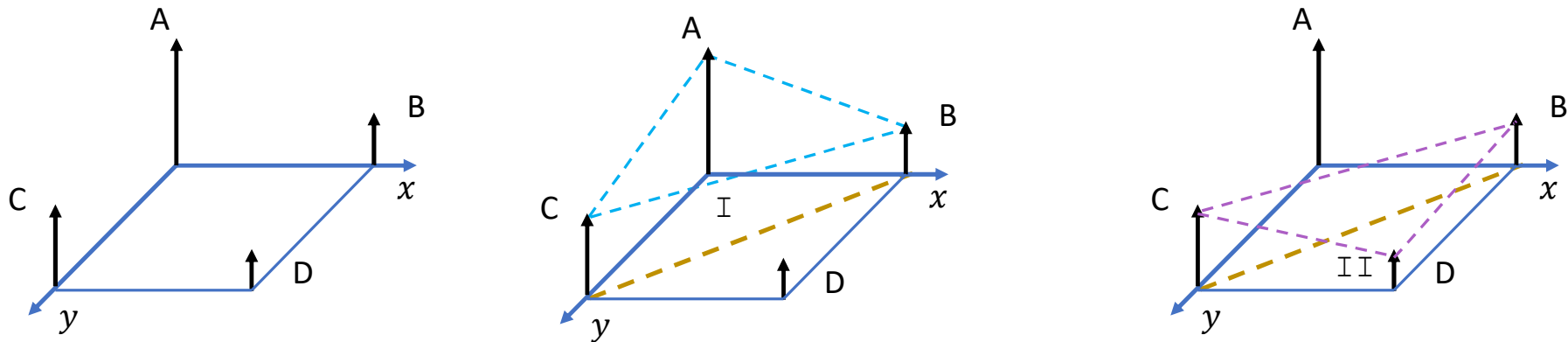


Equivalent to linearly connecting adjacent sample peaks



First order interpolation in 2D: Piecewise linear interpolation

Simple extension of linear interpolation to 2D does not hold because, in general, it is not possible to fit a plane through 4 adjacent samples



- For region I, points are linearly interpolated by the plane defined by pixels A, B, C
- For region II, points are linearly interpolated by the plane defined by pixels B, C, D

First order interpolation in 2D: Bilinear interpolation

- Computationally simpler than piecewise linear interpolation
- Involves 1D interpolation along rows, then 1D interpolation across columns

$$\Delta_x = \frac{x - a}{b - a}$$

$$0 \leq \Delta_x \leq 1$$

$$\Delta_y = \frac{y - c}{d - c}$$

$$0 \leq \Delta_y \leq 1$$

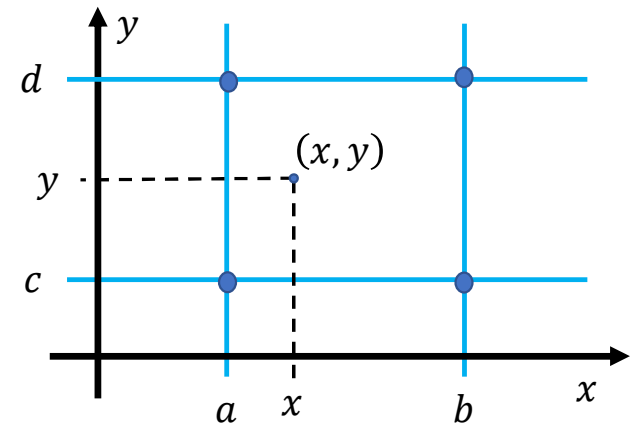
We can think of Δ_x as the fraction of the distance across from a to b

The interpolated surface is given by:

$$\hat{f}(x, y) = (1 - \Delta_x)(1 - \Delta_y)f(a, c) + (1 - \Delta_x)\Delta_y f(a, d) + \Delta_x(1 - \Delta_y)f(b, c) + \Delta_x\Delta_y f(b, d)$$

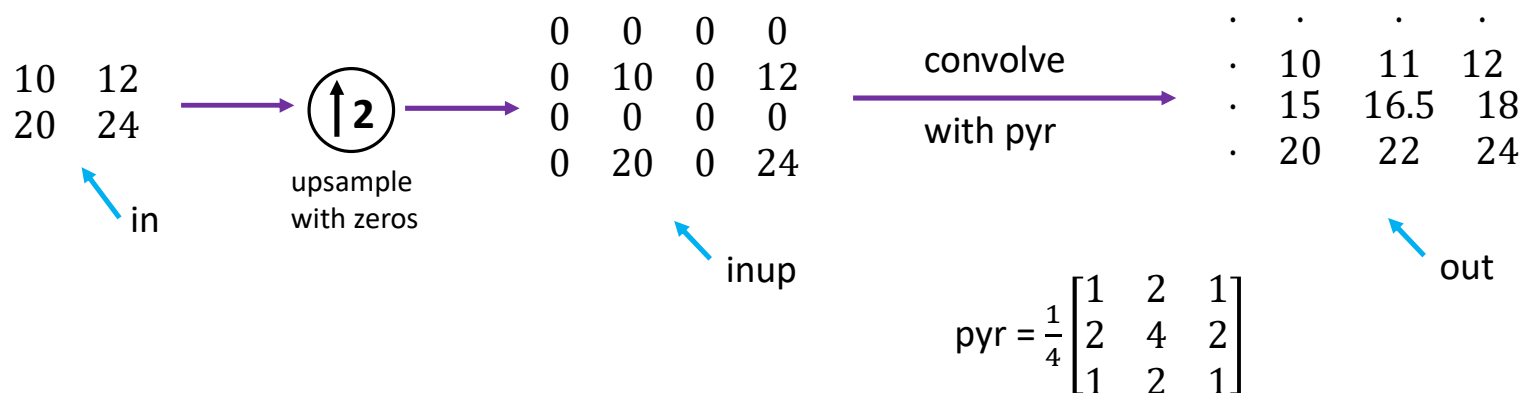
- Resultant interpolated surface generally non-planar

On a rectangular grid, consider the four discrete neighboring points which are known: $f(a, c)$, $f(b, c)$, $f(a, d)$, $f(b, d)$



Bilinear Interpolation

Equivalent to interpolation with a pyramid function



In Matlab:

- `out = imresize(in, 2, 'bilinear');`

Called bilinear interpolation because it is linear as a function of x when y is held fixed, and also linear as a function of y when x is held fixed

Bilinear interpolation example



Original

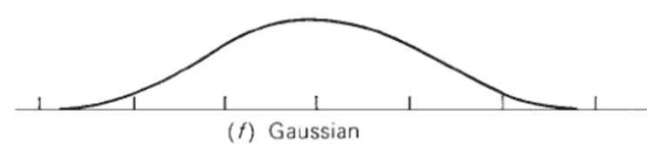
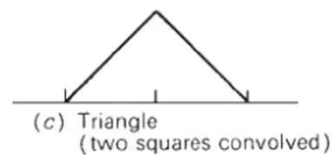
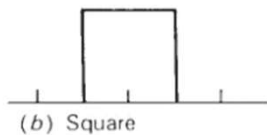
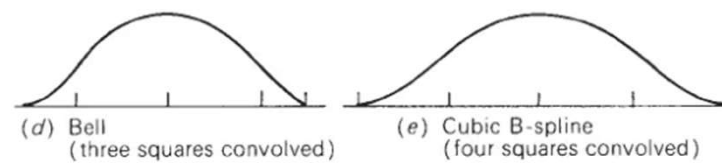
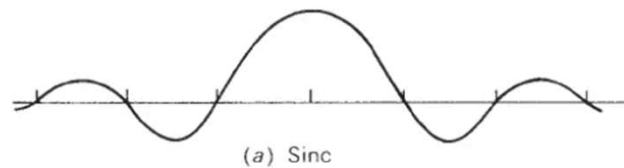


Sampled



Reconstructed

Convolution more and more box functions



$$R_0(x)$$

$$R_1(x) = R_0(x) * R_0(x)$$

Cubic B-spline:

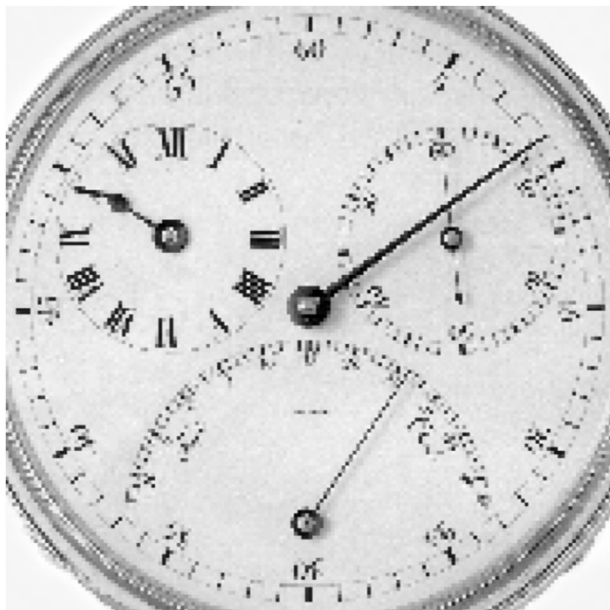
$$R_3(x) = R_1(x) * R_1(x) = \begin{cases} \frac{2}{3} + \frac{1}{2}|x|^3 - (x)^2 & \text{for } 0 \leq |x| \leq 1 \\ \frac{1}{6}(2 - |x|)^3 & \text{for } 1 < |x| \leq 2 \end{cases}$$

A *spline* is a piecewise polynomial with pieces that fit smoothly together

Interpolation comparison from Gonzalez and Woods (Fig 2.27)

Image reduced to 72dpi and interpolated back to original 930 dpi using three interpolation methods

Pixel replication



Blockiness (high frequency artifacts)

Bilinear

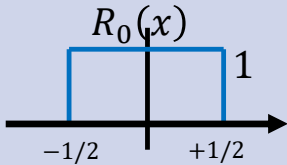
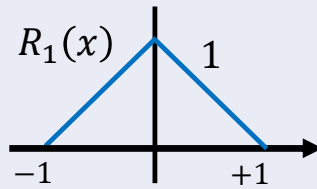
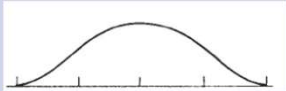
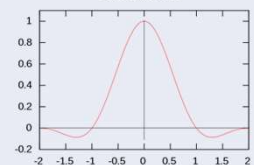


Bicubic



Slightly blurry

Interpolation Summary

1-D interpolation function	Graphical representation	2-D interpolation function
Rectangular (zero-order filter) $R_0(x)$		$R_0(x)R_0(y)$
Triangular filter (first order filter) $R_1(x) = R_0(x) * R_0(x)$		$R_1(x)R_1(y)$
n-order filter (n=2, quadratic) n=3 cubic spline, etc. $R_3(x) = R_1(x) * R_1(x)$		$R_3(x) R_3(y)$
Matlab also offers Lanczos kernels (windowed sinc fct)		$L(x)L(y)$

- Various tradeoffs among blurring, high frequency artifacts, and complexity
- Bicubic and Lanczos filters are commonly used
- Neural networks are excellent for this purpose